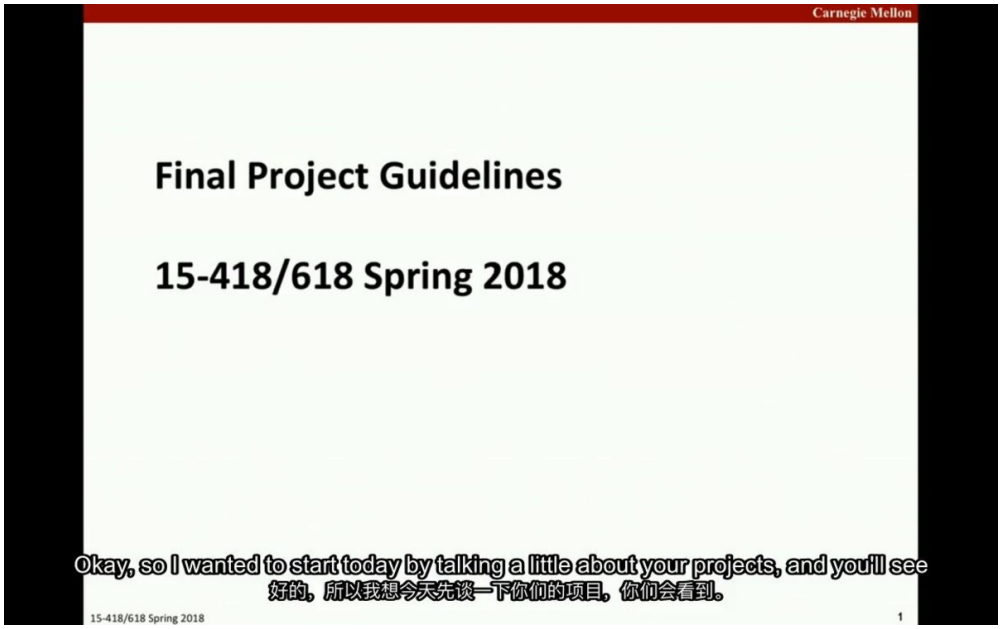
课程笔记

领域专用语言：生产力与性能的权衡

CMU 15-418/618 并行计算机架构与编程 · Lecture 24（Spring 2018）

lecture-to-notes

2026 年 10 月 29 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 20 分 1 秒
视频链接：`https://www.bilibili.com/video/BV16N4y197cm/`

目录

1 普通程序远未用尽硬件 **2**
1.1 基线是什么 2
1.2 其他语言留下了多少性能 3
1.3 本章小结 4

2 即使好代码也没用尽机器 **4**
2.1 并行带来的额外收益 4
2.2 但做起来比想起来难 5
2.3 本章小结 6

3 多个层次与多个模型 **6**
3.1 机器内部的层次 6
3.2 必须同时思考多个层次 7
3.3 本章小结 7

4 领域专用语言的策略 **8**
4.1 牺牲通用性 8
4.2 图作为抽象 9
4.3 本章小结 9

5 两个具体项目 **10**
5.1 本章小结 10

6 总结与延伸 **10**

1 普通程序远未用尽硬件

这一讲讨论领域专用语言。讲者的切入方式不是先讲语言设计，而是先说明一个观察：普通程序在普通机器上运行时，远远没有用尽可用的硬件能力。

1.1 基线是什么

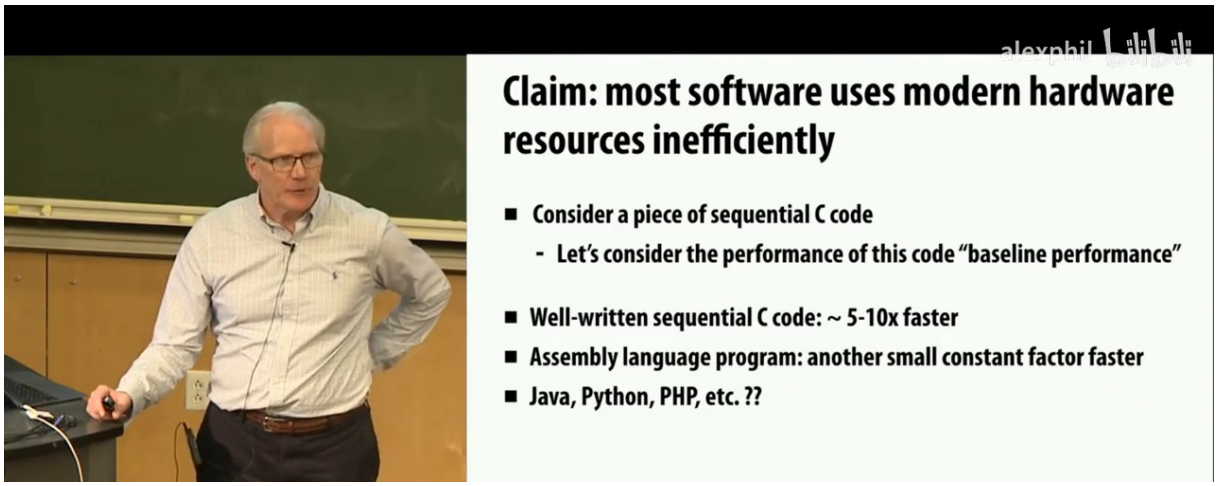


图 1: 论点：多数软件对现代硬件资源的使用效率很低[1]

讲者先明确基线：这里默认假设你用 C 或 C++ 编写，因此你已经处于编程层面上相当低的位置，有能力去压性能。

在这个前提下，传统智慧是：一段写得不错的串行程序，若有人坐下来认真调优、用好编译器而不走极端，可以获得大约 5 到 10 倍的性能。

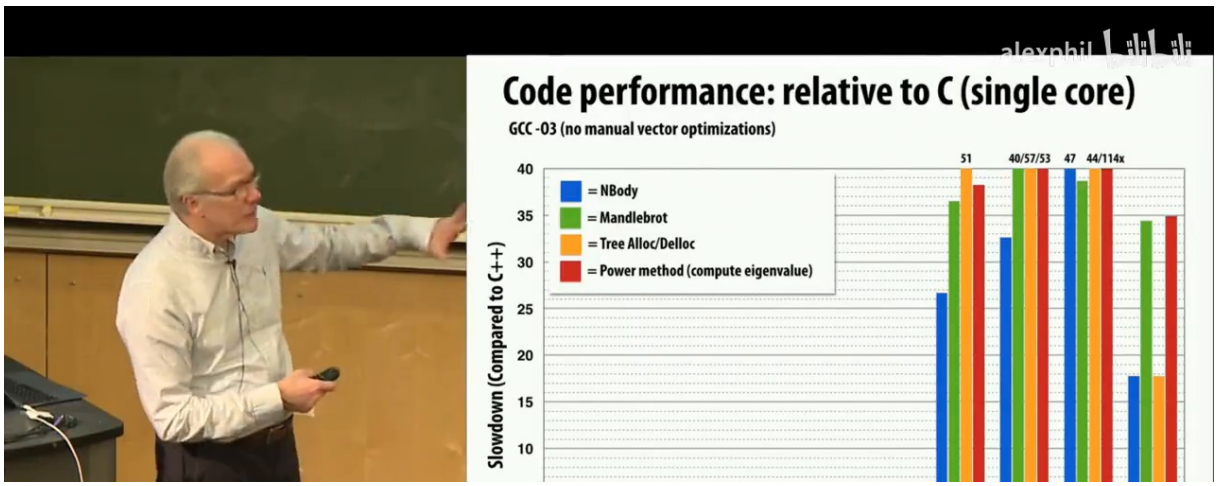


图 2: 各语言相对 C（单核，GCC -O3，无手工向量优化）的性能对比[2]

[1] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

[2] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

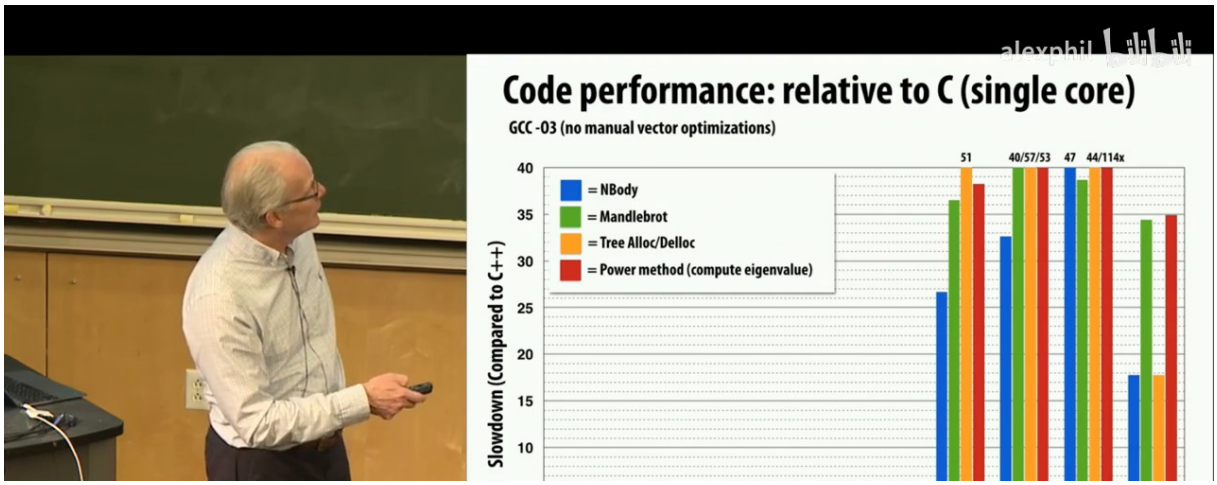


图 3: 性能对比续：纯解释型语言通常慢得多[3]

1.2 其他语言留下了多少性能

讲者接着问一个更有意思的问题：世界上大量程序是用别的语言写的，那些语言把多少性能留在了桌上?

他的答案是：很多。

他展示了一张对比图。基准是同一段程序用 C 实现、用相当常规的编译选项 GCC -O3（没有手工向量优化），其性能记为 1。其余各项表示相对 C 慢多少。

图中列出 Java、Scala（构建在 Java 之上）、C#（Windows 上的托管版 C）、Haskell（函数式语言）、Go（另一种托管语言）。讲者特别提到 Go 在这里跑得异常慢，而 JavaScript 的表现意外地相当不错。

而像 Python 这类纯解释型语言，通常要慢得多。

若把 C 实现记为基准 1，则其他语言可表示为相对倍数：

$$\text{speed}_{\text{lang}} = \frac{T_{\text{C}}}{T_{\text{lang}}}$$

- T_{C}：同一算法用 C 加常规优化编译后的耗时；
- T_{lang}：同一算法用目标语言实现的耗时；
- 该比值小于 1 表示比 C 慢，且往往小一个数量级甚至更多。

这个比值在写第一行代码时就基本确定，后续优化只能在其基础上改善。

语言选择本身就是一次性能决策

同一算法在不同语言下的性能差距可达数十倍，而这个差距在写代码的第一天就已确定。因此” 用什么语言实现” 不是风格问题，而是性能预算的分配问题。这也解释了为什么数值密集的核心循环常被下沉到 C 或 Fortran，而外围逻辑用高层语言。

[3]视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

1.3 本章小结

基线是写好的串行 C 代码；在此基础上认真调优可获得 5 到 10 倍。而其他语言相对 C 的性能落差很大：托管语言与函数式语言各有损耗，纯解释型语言慢得多。因此语言选择本身就是性能决策，差距在写第一行代码时就已确定。

2 即使好代码也没用尽机器

2.1 并行带来的额外收益

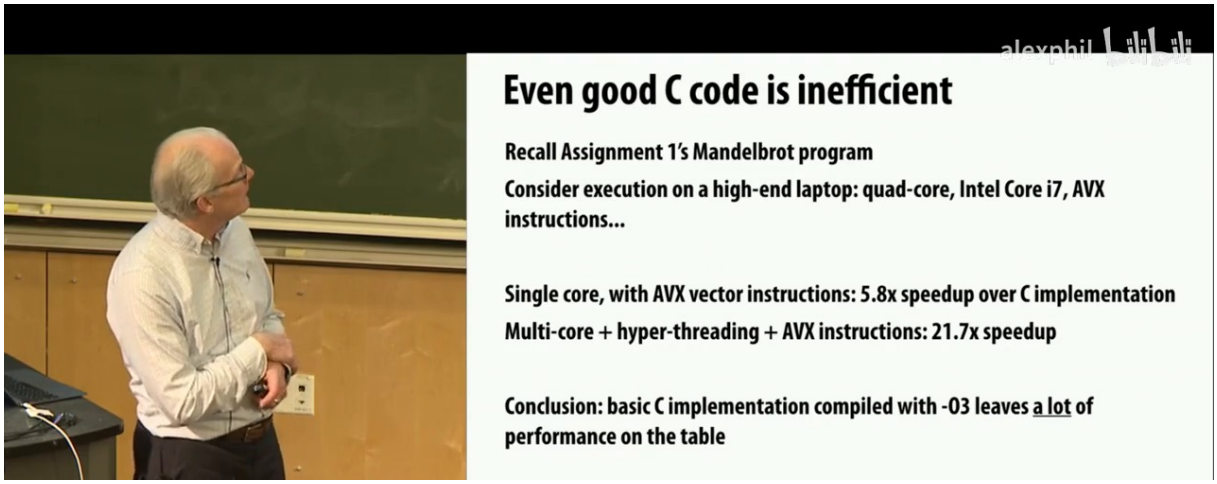


图 4: 即使好的 C 代码也效率不高：作业一的 Mandelbrot 程序[4]

讲者用课程作业一的 Mandelbrot 程序说明。通过利用超线程与多核，他们在 Gates 机器上获得了大约 20 倍的加速。

他补充说，即使在一台高端四核笔记本上，也能测到 20 倍。而这只是更好地利用了那台笔记本内置的资源，并不是把它映射到什么特别大的机器上。

所以他强调：相对于” 写好的串行 C 代码” 这个基线，同一台机器上仍有大量性能可以挖出来。

这两类收益可以分解相乘：

$$S_{\text{total}} = S_{\text{tune}} \times S_{\text{parallel}}$$

- S_{tune}：对串行代码本身调优的收益，典型为 5 到 10 倍；
- S_{parallel}：把已有并行资源用起来的收益，本例约 20 倍；
- 两者作用于不同层面，因此可以叠加而非互相替代。

$$\frac{T_{\text{naive C}}}{T_{\text{tuned parallel}}} \approx 20$$

- 分子：常规编译的串行 C 实现；
- 分母：用上 SIMD、多线程与超线程后的实现；
- 这个 20 倍并非来自更大的机器，而是来自把已有资源用起来。

[4]视频画面时间区间：00:27:30–00:27:45。若为裁剪图，时间区间仍对应原视频画面。

2.2 但做起来比想起来难

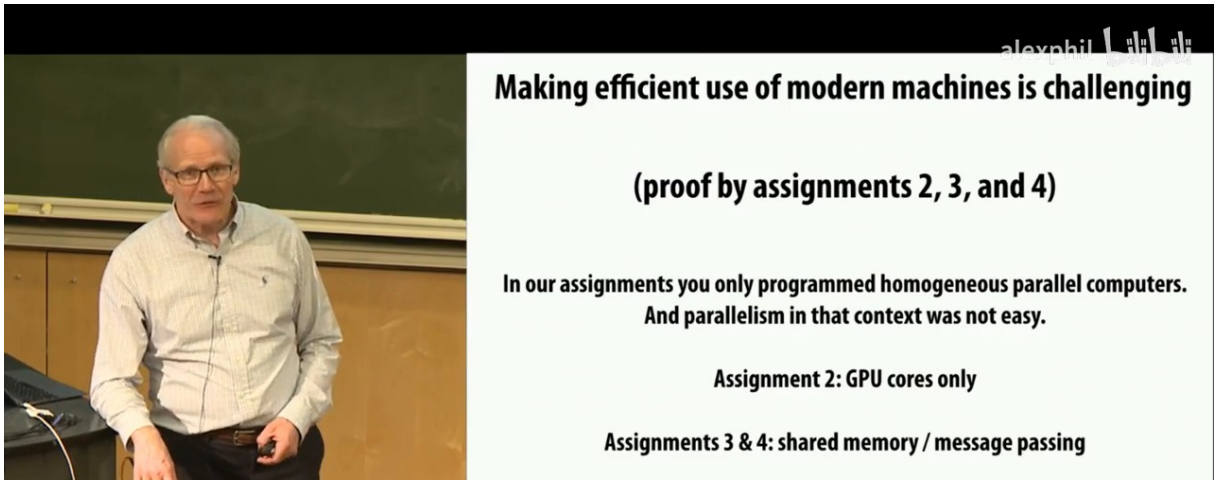


图 5: 高效利用现代机器很有挑战：作业二、三、四就是证明[5]

讲者说这是并行计算里一个特别好的案例。但他随即给出一个诚实的限定：通过他们的各项作业，我们发现实际做起来比想起来要难得多。

把一段程序改造成真正能利用并行机能力的形式，需要大量工作。

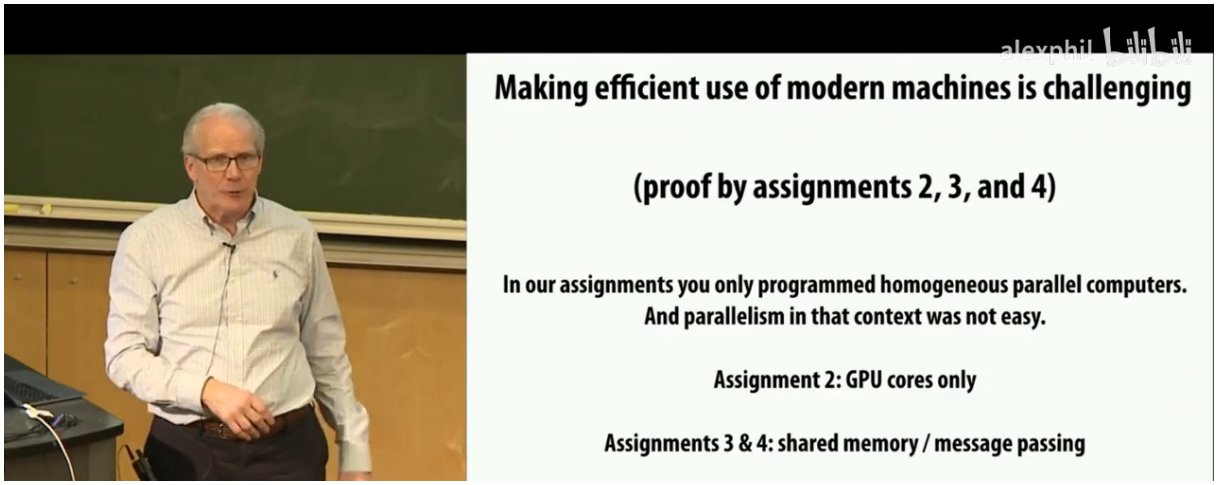


图 6: 不同并行模型几乎是不同的思维方式[6]

更难的是：映射到 GPU 与映射到 CPU，最终会形成很不同的代码风格。即使在 CPU 上，基于共享内存模型与基于消息传递模型，写法也差别很大。

所以从软件视角看，在这些机器之间移动时，你几乎需要不同的思维方式来推理程序。

”同一个算法”在不同并行模型下不是同一份代码

算法描述的是计算，而并行模型决定了如何组织数据、如何同步、如何划分工作。因此移植不是”改几个 API”，而是重写程序结构。这正是”生产力”问题在并行领域被放大的原因。

[5] 视频画面时间区间：00:28:30–00:28:45。若为裁剪图，时间区间仍对应原视频画面。

[6] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

2.3 本章小结

即使好的串行 C 代码也没用尽机器：利用超线程与多核可获得约 20 倍加速，而且这在四核笔记本上就能测到，并非需要大机器。但做起来比想起来难得多：把程序改造成能利用并行机能力需要大量工作，而 GPU 与 CPU、共享内存与消息传递各自要求不同的思维方式。

3 多个层次与多个模型

3.1 机器内部的层次

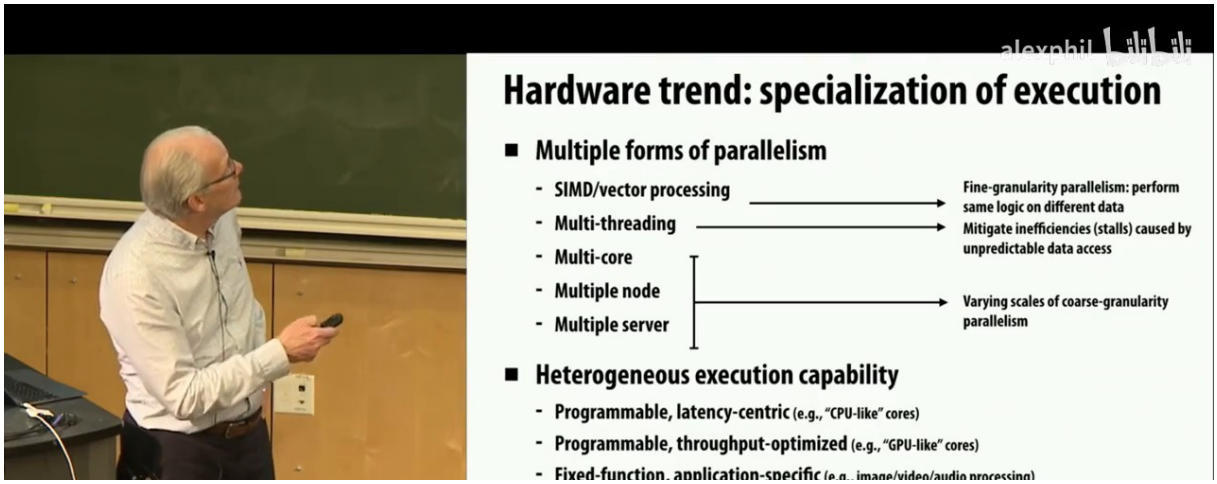


图 7: 硬件趋势：执行单元的专门化与多种并行形式[7]

讲者列举机器内部可挖掘的层次。

线程级，甚至超线程级：利用多个功能单元的未用潜力，隐藏因缓存缺失造成的访存延迟。

多核，甚至多服务器：上升到任务级或更粗粒度的并行，思考程序之间如何通信与交互。

所以即使在常规 CPU 内部，就已经有这么多不同层次可以挖掘。而加入各种异构的、某种程度上专门化的单元之后，这件事变得更棘手。

[7] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

3.2 必须同时思考多个层次

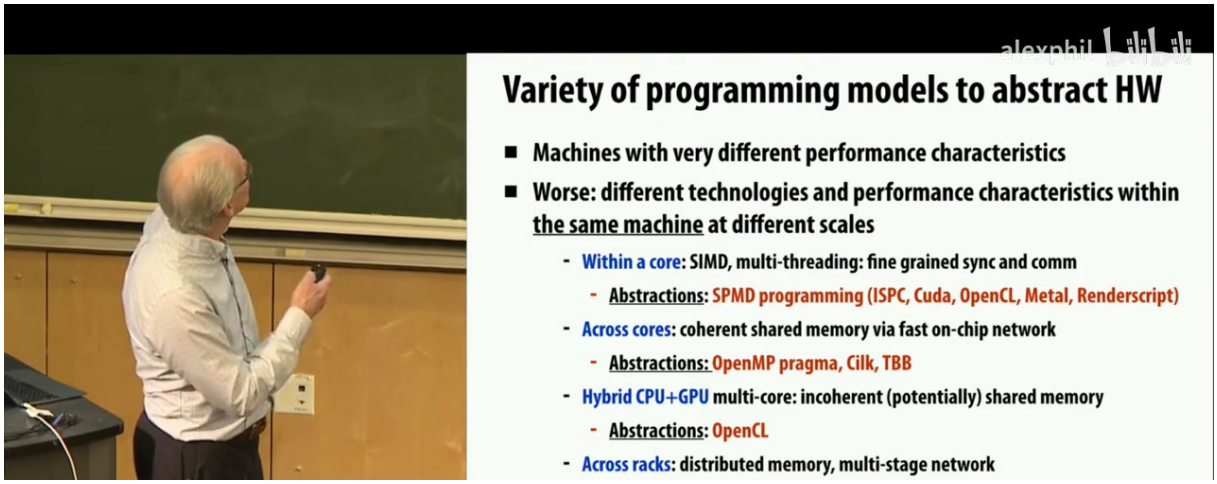


图 8: 为不同硬件提供多种编程模型：性能特征差异很大[8]

讲者的结论是：要真正拿到全部性能，你必须在多个层次上同时思考。

这对软件开发者与维护者已经是个问题，而且在变得更糟。

他还指出：若思考每个层次上最适合的编程模型，会发现它们各不相同。比如 SIMD 单元与 GPU 适合单程序多数据风格。

层次越多，”统一抽象”的代价越高

若为每一层各写一套代码，维护成本随层数线性增长；若强行用一套抽象覆盖所有层，抽象就必须足够通用，从而失去各层的优化机会。领域专用语言正是选择后一条路，但把优化交给编译器——它的成败取决于编译器能否在该领域内做对。

这些层次之间还有耦合，使问题进一步复杂。比如用多线程隐藏访存延迟时，线程数增加会加剧缓存争用；而缓存争用又会反过来降低每个线程的有效带宽。因此在某一层上做出的选择，常常改变另一层的代价结构。这意味着「逐层独立优化」并不成立——必须把几层放在一起考虑，这也解释了为什么并行程序的调优往往比串行程序更需要全局视角。讲者提到，作业里同学们常见的失败模式正是在一层上优化、却让另一层退化。

3.3 本章小结

机器内部有多层可挖掘：线程级与超线程用于隐藏访存延迟，多核与多服务器可上升到任务级，加入异构专用单元后更棘手。要拿到全部性能必须在多个层次上同时思考，而每层适合的编程模型不同。这使统一抽象与逐层优化成为一对必须权衡的代价。

[8] 视频画面时间区间：00:31:30–00:31:45。若为裁剪图，时间区间仍对应原视频画面。

4　领域专用语言的策略

4.1　牺牲通用性

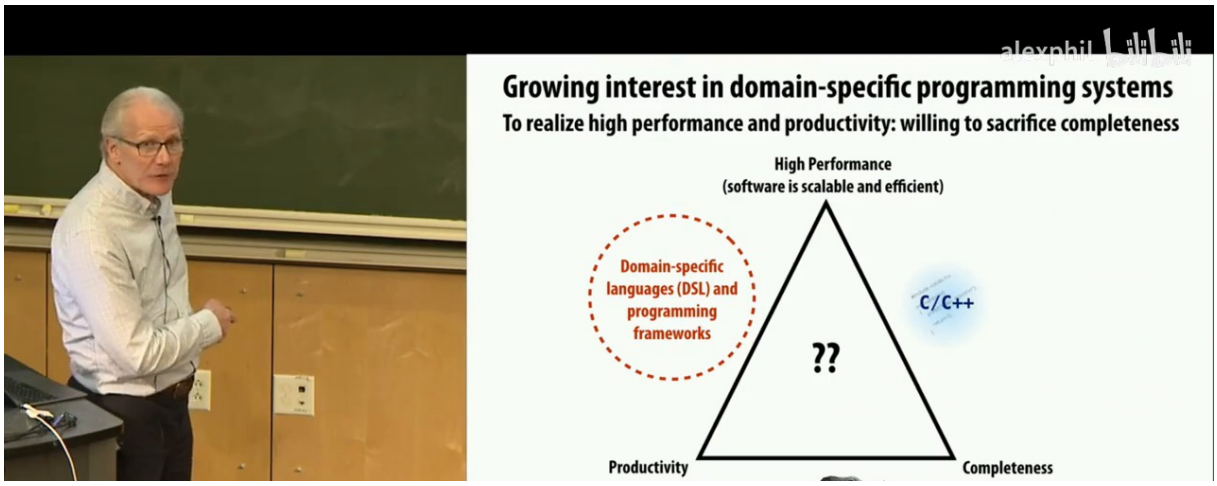


图 9: 领域专用编程系统的兴趣在增长：兼顾高性能与生产力[9]

讲者把领域专用语言的策略说得很直接：为了实现高性能与生产力的兼得，就要牺牲通用性。

也就是说，不追求一个能表达任何计算的通用语言，而是针对某一类问题设计语言，把该领域的知识编码进编译器与运行时。

领域专用语言的赌注是”领域内的规律足够稳定”

它假设该领域的计算模式足够固定，因此可以预先设计好优化。若领域内的需求快速变化，专用语言的优化就会迅速过时。因此这类语言最适合那些问题形态多年不变、而硬件不断变化的领域——把变化的负担从程序员转移到编译器。

这里有一个值得注意的取舍：领域专用语言把优化责任从程序员转移给编译器，但编译器只能优化它「理解」的模式。因此语言设计者必须预先决定哪些模式值得支持。支持得越多，编译器越难对每种情况都做到最优；支持得越少，语言的适用范围就越窄。这个张力决定了专用语言的成败——它必须在「足够覆盖领域」与「足够专注以优化」之间找到平衡点。

[9] 视频画面时间区间：00:35:00–00:35:15。若为裁剪图，时间区间仍对应原视频画面。

4.2 图作为抽象

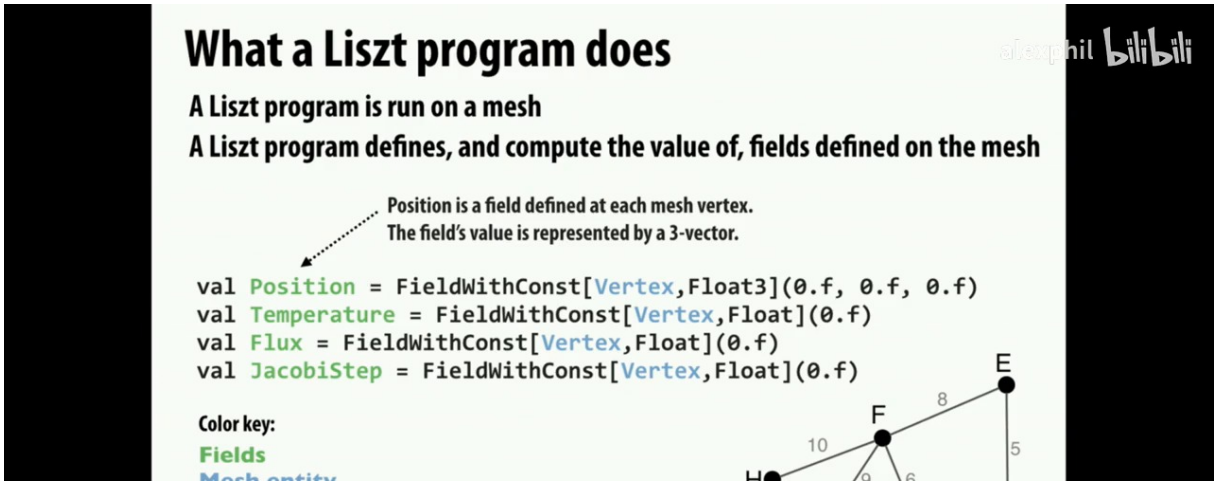


图 10: Liszt 程序在网格上运行，定义并计算面与顶点的量[10]

讲者举图为例：图本身就是一种自然的抽象，节点表示实体，边表示两个不同实体之间的关系。因此可以在这些不同的抽象层次上表达计算。

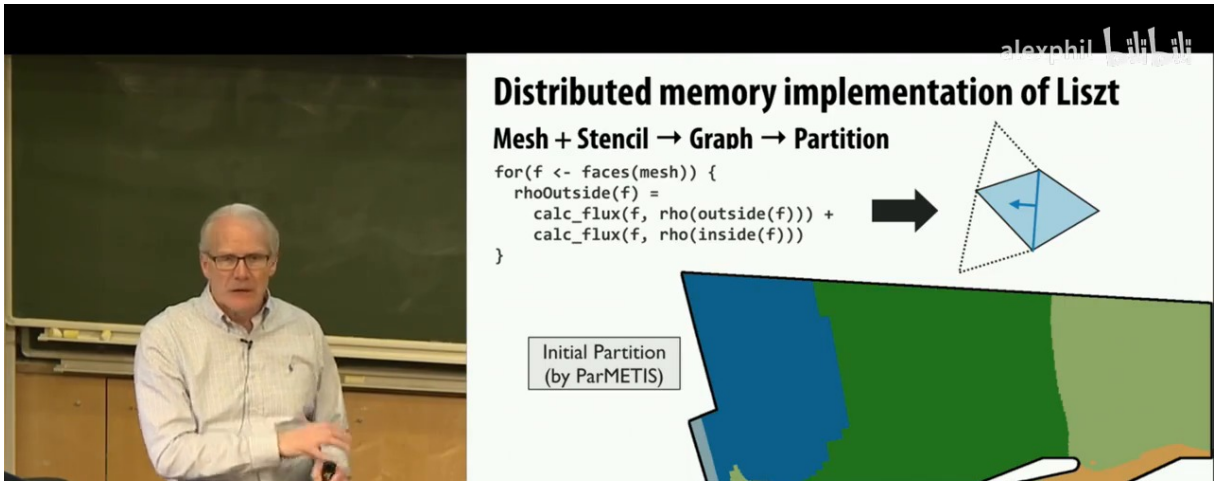


图 11: 网格通常也不是均匀的：Liszt 的分布式实现把网格转成图再分区[11]

他指出一个实际细节：网格通常也不是均匀的。因此这类语言需要处理不规则性，而这正是它比通用语言更有优势的地方——不规则性在领域内是已知的，可以被编译器预先考虑。

4.3 本章小结

领域专用语言的策略是牺牲通用性换取高性能与生产力：针对一类问题设计语言，把领域知识编码进编译器与运行时。图是自然的抽象（节点是实体、边是关系），而网格通常并不均匀，处理这种不规则性正是专用语言的优势所在。

[10] 视频画面时间区间：00:40:00–00:40:15。若为裁剪图，时间区间仍对应原视频画面。
[11] 视频画面时间区间：00:50:00–00:50:15。若为裁剪图，时间区间仍对应原视频画面。

5 两个具体项目

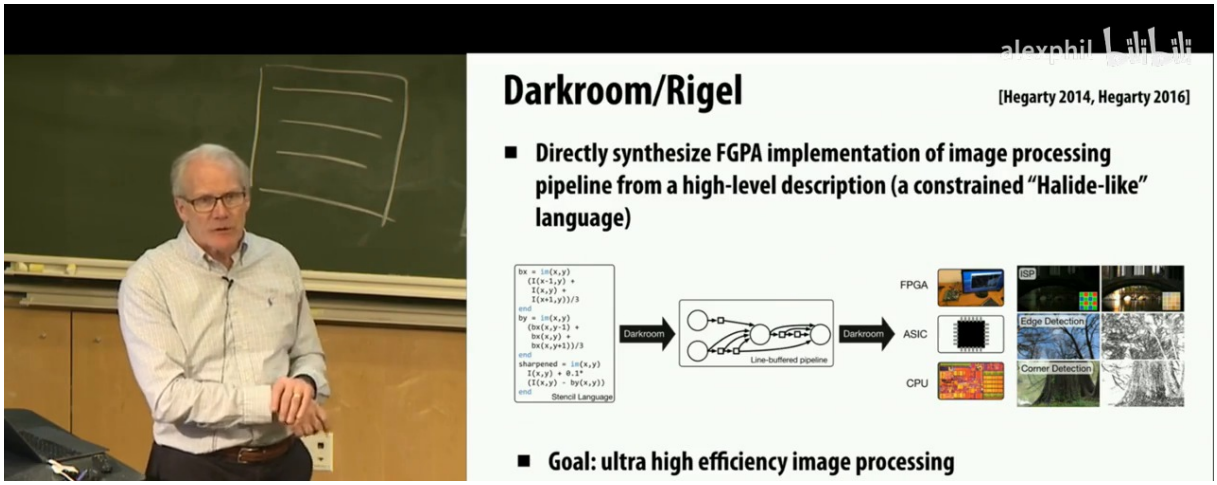


图 12: 两个具体项目：Darkroom/Rigel 直接合成 FPGA 实现[12]

讲者在结尾介绍了两个具体项目。其中之一是 Darkroom 与 Rigel：直接把图像处理流程合成为 FPGA 实现。

另一个的思路是把计算函数表达为图上的运算。讲者说这类项目并非独创，已有各种类似的工作，但这两个项目在" 映射到我们所关心的计算机" 这一点上特别有力。

专用化的终点是硬件生成

当领域语言足够受限时，编译器不仅能生成优化代码，还能直接生成硬件描述。这把这门语言的" 目标机器" 从处理器扩展到了 FPGA 或专用电路，从而绕过通用处理器的能效限制。这是第 23 讲异构与专用化思路的极端形式。

5.1 本章小结

两个具体项目展示了专用化的两个方向：Darkroom/Rigel 直接把图像处理流程合成为 FPGA 实现；另一个把计算函数表达为图上的运算。两者都在" 映射到目标机器" 这一点上特别有力。专用化的极端形式是直接生成硬件，从而绕过通用处理器的能效限制。

6 总结与延伸

讲者在结尾说，这类思路并非独创，已有各种类似工作，但把它们映射到我们所关心的计算机上这件事特别有价值。

把全讲串起来，这是一条从" 硬件远未被用尽" 到" 用领域专用语言改变这一状况" 的路径。基线是写好的串行 C 代码，在此基础上认真调优可获得 5 到 10 倍；而其他语言相对 C 的落差很大，纯解释型语言慢得多，因此语言选择本身就是性能决策。即使好的 C 代码也没用尽机器：利用超线程与多核可获得约 20 倍，而且这在四核笔记本上就能测到。但做起来比想起来难得多：把程序改造成能利用并行机能力需要大量工作，而 GPU 与 CPU、共享内存与消息传递各自要求不同的思维方式。机器内部有多层可挖掘，每层适合的编程模型不同，因此统一抽象与逐层优化是一对必须权衡

[12] 视频画面时间区间：01:19:00–01:19:15。若为裁剪图，时间区间仍对应原视频画面。

的代价。领域专用语言的策略是牺牲通用性：针对一类问题设计语言，把领域知识编码进编译器与运行时。图是自然的抽象，而网格通常并不均匀，处理这种不规则性正是专用语言的优势。最后，两个具体项目展示了专用化的两个方向，其极端形式是直接生成硬件。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- ”20 倍” 与”5 到 10 倍” 是两个不同性质的数字。前者来自把已有并行资源用起来，后者来自对串行代码本身的优化。它们可以叠加，而不是互相替代。
- 语言之间的性能落差在写第一行代码时就已确定，这一点常被低估。因此” 用高层语言写原型、把热点下沉到 C” 是常见策略，但下沉本身有接口与维护成本。
- 领域专用语言的赌注是领域规律足够稳定。若领域需求快速变化，预置的优化会迅速过时。因此它适合问题形态多年不变而硬件不断变化的场景。
- 专用化的终点可以是硬件生成。当领域足够受限时，编译器能直接输出硬件描述，从而绕过通用处理器的能效限制。这是异构思路的极端形式，也是专用语言与硬件设计的交汇点。

这堂课的边界在于它是动机与项目介绍：讲者用大量时间讨论项目要求与选题，DSL 部分给出的是策略与两个例子，未深入语言的具体设计或编译技术。

两个可以直接从本讲数据提出的下一步问题：其一，既然语言选择决定了数十倍的性能落差，那么在什么规模的程序中” 换语言重写” 比” 在原语言内优化” 更划算？其二，领域专用语言把优化责任转移给编译器，那么当编译器无法处理某个新模式时，程序员还有什么手段？